

Machine Learning

Core Concepts Handout

SUMMER 2026 — BSIT, 3RD SEMESTER

Supervised Learning • Unsupervised Learning • Model Evaluation • Neural Networks •
Deep Learning • Reinforcement Learning • Ethics

Contents

1	Introduction to Machine Learning	4
1.1	What Is Machine Learning?	4
1.2	A Brief History	4
1.3	Machine Learning vs. Traditional Programming	4
2	Types of Machine Learning	5
2.1	Supervised, Unsupervised, and Reinforcement Learning	5
2.2	The Machine Learning Workflow	5
3	Data Preprocessing and Feature Engineering	6
3.1	Handling Missing Data and Outliers	6
3.2	Feature Scaling and Encoding	6
3.3	Feature Selection and Engineering	6
4	Linear and Logistic Regression	6
4.1	Simple and Multiple Linear Regression	7
4.2	Gradient Descent	7
4.3	Logistic Regression for Classification	7
5	Classification Algorithms I: Distance and Probability Based	7
5.1	K-Nearest Neighbors (KNN)	8
5.2	Naïve Bayes Classifier	8
6	Classification Algorithms II: Trees and Margins	8
6.1	Decision Trees	9
6.2	Support Vector Machines (SVM)	9
7	Ensemble Learning	9
7.1	Bagging and Random Forests	10
7.2	Boosting	10

8	Unsupervised Learning: Clustering and Dimensionality Reduction	11
8.1	K-Means Clustering	11
8.2	Hierarchical Clustering	11
8.3	DBSCAN	11
8.4	Principal Component Analysis (PCA)	11
9	Model Evaluation and Validation	12
9.1	Train/Validation/Test Splits and Cross-Validation	12
9.2	Classification Metrics	12
9.3	Regression Metrics	13
10	Overfitting, Underfitting, and Regularization	13
10.1	The Bias-Variance Tradeoff	13
10.2	L1/L2 Regularization	13
10.3	Hyperparameter Tuning	14
11	Introduction to Neural Networks	14
11.1	The Perceptron and Multi-Layer Perceptron	14
11.2	Activation Functions	14
11.3	Backpropagation	15
12	Introduction to Deep Learning	15
12.1	Convolutional Neural Networks (CNNs)	15
12.2	Recurrent Neural Networks (RNNs)	15
13	Reinforcement Learning Basics	16
13.1	Agents, Environments, and Rewards	16
13.2	Q-Learning	16
14	The Machine Learning Pipeline and Deployment	16
14.1	From Notebook to Production	16
14.2	Model Monitoring and Drift	17
15	Ethical and Responsible Machine Learning	17
16	Core Mathematical Foundations for Machine Learning	18
16.1	Linear Algebra Essentials	18
16.2	Probability and Statistics Essentials	18
16.3	Calculus and Optimization Essentials	18
17	Exam-Oriented Worked Examples	18
18	Quick Revision Glossary	19
19	Question Bank Blueprint (80 Marks Support)	22
19.1	Important Theory Questions	22
19.2	Important Numerical/Analytical Questions	23
19.3	Sample MCQs with Answer Key	23

20 Summary

24

1 Introduction to Machine Learning

Machine Learning

Machine Learning (ML) is the field of study that gives computers the ability to learn patterns from data and improve their performance on a task through experience, without being explicitly programmed with fixed rules for every case.

Why Machine Learning Matters

ML powers spam filters, recommendation systems, fraud detection, medical diagnosis support, and self-driving cars. For BSIT students, ML is the core technology behind most modern “smart” applications and a foundation for further study in AI and data science.

1.1 What Is Machine Learning?

- **Arthur Samuel (1959):** “The field of study that gives computers the ability to learn without being explicitly programmed.”
- **Tom Mitchell (1997):** A program learns from experience E with respect to task T and performance measure P , if its performance at T , measured by P , improves with E .
- **Core idea:** Instead of hand-coding rules, an ML algorithm infers a model (a function) directly from example data.

1.2 A Brief History

- **1950s:** Alan Turing poses “Can machines think?”; Arthur Samuel writes a self-improving checkers program.
- **1957:** Frank Rosenblatt invents the Perceptron, the first trainable neural network.
- **1980s–1990s:** Decision trees, SVMs, and statistical learning theory mature.
- **2006–2012:** “Deep learning” re-emerges as deeper neural networks and GPUs make large-scale training practical.
- **2012–present:** Deep learning dominates computer vision, NLP, and speech; large language models emerge in the 2020s.

1.3 Machine Learning vs. Traditional Programming

Aspect	Traditional Programming	Machine Learning
Input	Rules + Data	Data + Desired Output (labels)
Output	Direct answer	A model / a set of rules
Adapting to Change	Requires rewriting rules manually	Retrain on new data
Best Suited For	Well-defined, deterministic logic	Complex patterns hard to hand-code (images, text)

2 Types of Machine Learning

Learning Paradigm

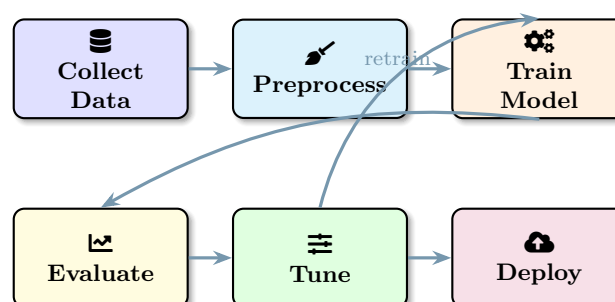
A learning paradigm defines what kind of feedback (if any) an algorithm receives from data while learning: labeled outputs, no labels, or rewards from interacting with an environment.

2.1 Supervised, Unsupervised, and Reinforcement Learning

Type	Feedback Signal	Example Task
Supervised	Labeled input-output pairs	Spam detection, house price prediction
Unsupervised	No labels, only raw data	Customer segmentation, anomaly detection
Semi-Supervised	A few labels, mostly unlabeled data	Medical imaging with limited annotated scans
Reinforcement	Rewards/penalties from an environment	Game playing, robot control

- **Supervised Learning:** Learns a mapping from inputs x to known outputs y ; further split into **regression** (continuous y) and **classification** (discrete y).
- **Unsupervised Learning:** Finds hidden structure in unlabeled data, e.g., clustering or dimensionality reduction.
- **Reinforcement Learning:** An agent learns a policy by taking actions in an environment and receiving rewards.

2.2 The Machine Learning Workflow



- **Collect Data:** Gather representative, sufficiently large data for the task.
- **Preprocess:** Clean, transform, and engineer features (covered next).
- **Train Model:** Fit an algorithm's parameters to the training data.
- **Evaluate:** Measure performance on unseen (validation/test) data.
- **Tune:** Adjust hyperparameters or features and retrain if performance is inadequate.
- **Deploy:** Put the trained model into production for real-world use.

3 Data Preprocessing and Feature Engineering

Feature

A feature is an individual measurable property of the data used as input to a machine learning model, e.g., a customer's age, income, or purchase count.

3.1 Handling Missing Data and Outliers

- **Missing Data:** Handled by deletion (dropping rows/columns) or imputation (filling with mean, median, mode, or a predicted value).
- **Outliers:** Extreme values that can distort model training; detected via boxplots, z-scores, or the IQR rule, then removed, capped, or transformed.
- **Data Leakage:** A common pitfall where information from outside the training set (e.g., from the test set) leaks into training, giving an unrealistically optimistic evaluation.

3.2 Feature Scaling and Encoding

Why Scale Features?

Algorithms that rely on distances (KNN, K-Means, SVM) or gradient descent (linear/logistic regression, neural networks) are sensitive to feature scale — a feature ranging 0–100,000 can dominate one ranging 0–1 unless both are rescaled.

$$\text{Min-Max Scaling: } x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad \text{Standardization: } x' = \frac{x - \mu}{\sigma}$$

- **One-Hot Encoding:** Converts a categorical variable (e.g., “color”: red/green/blue) into binary indicator columns.
- **Label Encoding:** Maps categories to integers (0, 1, 2, ...); suitable only when categories have a natural order.

3.3 Feature Selection and Engineering

- **Feature Engineering:** Creating new, more informative features from raw data, e.g., extracting “day of week” from a timestamp.
- **Feature Selection:** Choosing the most relevant subset of features (e.g., via correlation analysis or feature importance) to reduce noise and overfitting.
- **Curse of Dimensionality:** As the number of features grows, data becomes sparse in the feature space, making patterns harder to learn reliably.

4 Linear and Logistic Regression

Linear Regression

Linear regression models the relationship between input features and a continuous target variable as a weighted linear combination, fitting the line (or hyperplane) that minimizes prediction error.

4.1 Simple and Multiple Linear Regression

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

- **Simple Linear Regression:** One input feature, $\hat{y} = w_0 + w_1x$.
- **Multiple Linear Regression:** Several input features combined linearly.
- **Cost Function:** Mean Squared Error (MSE) measures the average squared gap between predictions and actual values:

$$J(w) = \frac{1}{m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

4.2 Gradient Descent

💡 How Gradient Descent Works

Gradient descent iteratively adjusts the weights w in the direction that most reduces the cost function $J(w)$, controlled by a learning rate α :

$$w := w - \alpha \frac{\partial J(w)}{\partial w}$$

Too large an α can overshoot the minimum; too small makes training slow.

4.3 Logistic Regression for Classification

📖 Logistic Regression

Logistic regression predicts the probability that an input belongs to a class by passing a linear combination of features through the sigmoid function; despite its name, it is a **classification** algorithm.

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = w_0 + w_1x_1 + \cdots + w_nx_n$$

- If $\hat{p} \geq 0.5$, predict class 1; otherwise predict class 0 (threshold is tunable).
- Trained by minimizing **log loss** (cross-entropy), not MSE.

5 Classification Algorithms I: Distance and Probability Based

📖 Classification

Classification is the supervised learning task of predicting a discrete class label for an input, e.g., “spam” vs. “not spam.”

5.1 K-Nearest Neighbors (KNN)

💡 How KNN Classifies

To classify a new point, KNN finds the k closest training points (usually by Euclidean distance) and assigns the majority class among them — there is no explicit training phase (a “lazy learner”).

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

- **Choosing k :** Small k is sensitive to noise (overfits); large k oversmooths (underfits).
- **Downside:** Slow at prediction time on large datasets, since it must compute distances to all training points.

5.2 Naive Bayes Classifier

📖 Naive Bayes

Naive Bayes applies Bayes’ Theorem, assuming all features are conditionally independent given the class — an assumption that is rarely exactly true, but works remarkably well in practice.

$$P(\text{class} \mid \text{features}) = \frac{P(\text{features} \mid \text{class}) P(\text{class})}{P(\text{features})}$$

🔗 Illustrative Example: Spam Classification

Given $P(\text{spam}) = 0.4$, $P(\text{“free”} \mid \text{spam}) = 0.7$, $P(\text{“free”} \mid \text{ham}) = 0.1$, $P(\text{ham}) = 0.6$:

$$P(\text{spam}) \times P(\text{“free”} \mid \text{spam}) = 0.4 \times 0.7 = 0.28$$

$$P(\text{ham}) \times P(\text{“free”} \mid \text{ham}) = 0.6 \times 0.1 = 0.06$$

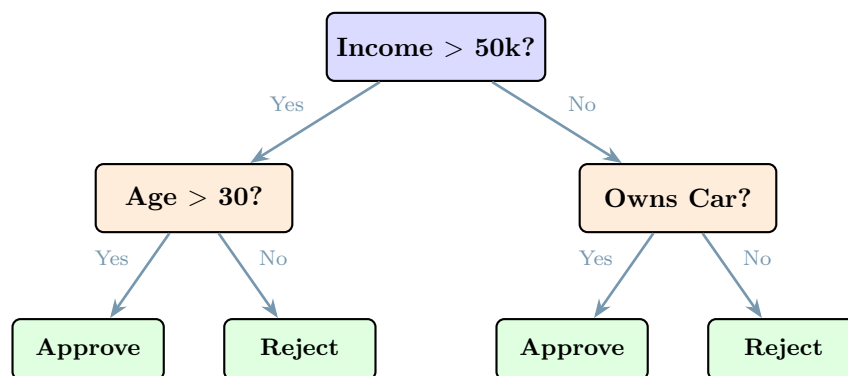
Since $0.28 > 0.06$, classify the email as **spam**.

6 Classification Algorithms II: Trees and Margins

📖 Decision Tree

A decision tree splits the data repeatedly on feature thresholds, forming a tree of yes/no questions that leads to a class label (or value) at each leaf.

6.1 Decision Trees



💡 Choosing the Best Split

At each node, the algorithm picks the feature/threshold that most reduces impurity, measured by **Entropy** or the **Gini Index**:

$$\text{Entropy}(S) = - \sum_i p_i \log_2 p_i, \quad \text{Gini}(S) = 1 - \sum_i p_i^2$$

Information Gain = Entropy(parent) – weighted-average Entropy(children).

- **Pruning:** Limiting tree depth or removing branches to reduce overfitting.
- **Advantage:** Easy to interpret and visualize; handles both numeric and categorical features.

6.2 Support Vector Machines (SVM)

📖 Support Vector Machine

An SVM finds the hyperplane that separates two classes with the **maximum margin** — the largest possible distance to the nearest points of each class (the “support vectors”).

- **Kernel Trick:** Maps data into a higher-dimensional space (e.g., via a polynomial or RBF kernel) so that classes that are not linearly separable in the original space become separable.
- **Soft Margin:** A tunable penalty (C) allows some misclassified points, trading margin width for fewer errors.

💡 Comparing the Classification Algorithms Covered So Far

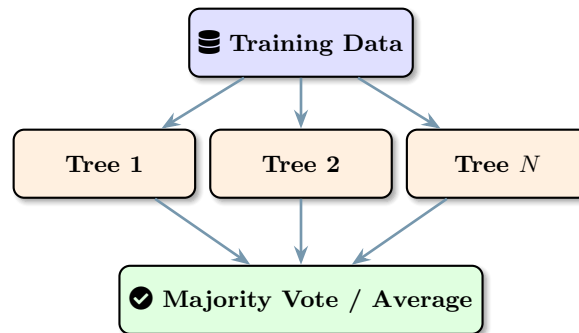
Algorithm	Strength	Weakness
KNN	Simple, no training phase	Slow at prediction time, sensitive to scale
Naive Bayes	Fast, works well with text/high dimensions	Independence assumption rarely holds exactly
Decision Tree	Easy to interpret and visualize	Prone to overfitting if unpruned
SVM	Effective in high-dimensional spaces	Slower to train on very large datasets

7 Ensemble Learning

Ensemble Learning

Ensemble learning combines predictions from multiple models to produce a result that is typically more accurate and robust than any single model alone.

7.1 Bagging and Random Forests



- **Bagging (Bootstrap Aggregating):** Trains many models on different random samples (with replacement) of the data, then averages/votes their predictions to reduce variance.
- **Random Forest:** Bagging applied to decision trees, with each tree also using a random subset of features at each split — reduces correlation between trees.

7.2 Boosting

How Boosting Works

Boosting trains models **sequentially**, where each new model focuses on correcting the errors of the previous ones (e.g., by up-weighting misclassified samples), then combines them into a strong final predictor.

- **AdaBoost:** Re-weights misclassified training examples so subsequent weak learners focus on the hardest cases.
- **Gradient Boosting:** Each new model is trained to predict the *residual error* of the ensemble so far (e.g., XGBoost, LightGBM).

Aspect	Bagging	Boosting
Training Style	Parallel, independent models	Sequential, each model corrects prior errors
Goal	Reduce variance	Reduce bias
Overfitting Risk	Lower	Higher if not tuned carefully
Example	Random Forest	AdaBoost, XGBoost, LightGBM

8 Unsupervised Learning: Clustering and Dimensionality Reduction

Clustering

Clustering groups data points so that points in the same group (cluster) are more similar to each other than to points in other groups, with no predefined labels.

8.1 K-Means Clustering

The K-Means Algorithm

1. Choose k (the number of clusters) and randomly initialize k centroids.
2. **Assign:** Assign each point to its nearest centroid.
3. **Update:** Recompute each centroid as the mean of the points assigned to it.
4. Repeat Assign/Update until centroids stop changing (convergence).

- **Choosing k :** The **Elbow Method** plots within-cluster variance against k and looks for the “elbow” where improvement slows.
- **Limitation:** Assumes roughly spherical, similarly-sized clusters; sensitive to centroid initialization.

8.2 Hierarchical Clustering

- **Agglomerative (bottom-up):** Starts with every point as its own cluster, then repeatedly merges the two closest clusters.
- **Dendrogram:** A tree diagram showing the order and distance at which clusters were merged; cutting it at a chosen height yields a specific number of clusters.

8.3 DBSCAN

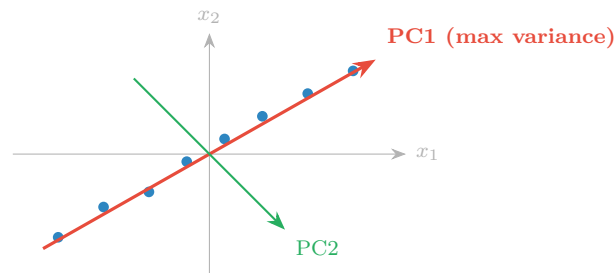
DBSCAN

Density-Based Spatial Clustering (DBSCAN) groups points that are closely packed together (dense regions), marking points in low-density regions as **noise/outliers**, and — unlike K-Means — does not require specifying the number of clusters in advance.

8.4 Principal Component Analysis (PCA)

PCA

PCA is a dimensionality reduction technique that projects data onto a smaller number of new axes (**principal components**) that capture the maximum possible variance in the data.



- **Principal Components:** Orthogonal directions ranked by how much variance of the data they explain, derived from the eigenvectors of the data's covariance matrix.
- **Why Reduce Dimensions:** Speeds up training, reduces storage, fights the curse of dimensionality, and enables 2D/3D visualization of high-dimensional data.
- **Trade-off:** Some information (variance) is inevitably lost when discarding lower-ranked components.

9 Model Evaluation and Validation

Why Evaluate?

Model evaluation estimates how well a trained model will perform on new, unseen data — the entire point of building a model in the first place.

9.1 Train/Validation/Test Splits and Cross-Validation

- **Training Set:** Used to fit the model's parameters.
- **Validation Set:** Used to tune hyperparameters and select between models.
- **Test Set:** Used only once, at the very end, to report final, unbiased performance.
- **k-Fold Cross-Validation:** Splits data into k folds, trains on $k - 1$ folds and validates on the remaining fold, rotating k times and averaging the results — more robust than a single split.
- **Stratified k-Fold:** Ensures each fold preserves the overall class proportions, important for imbalanced classification datasets.
- **Leave-One-Out Cross-Validation (LOOCV):** The extreme case of k -fold where k equals the number of samples; thorough but computationally expensive.

9.2 Classification Metrics

TP Predicted Yes, Actual Yes	FP Predicted Yes, Actual No
FN Predicted No, Actual Yes	TN Predicted No, Actual No

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

💡 When Accuracy Is Misleading

On an imbalanced dataset (e.g., 95% non-fraud, 5% fraud), a model that always predicts “non-fraud” scores 95% accuracy while catching zero fraud — Precision, Recall, and F_1 reveal this failure; **ROC-AUC** summarizes performance across all classification thresholds.

9.3 Regression Metrics

$$MAE = \frac{1}{m} \sum_{i=1}^m |y^{(i)} - \hat{y}^{(i)}|, \quad MSE = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

$$RMSE = \sqrt{MSE}, \quad R^2 = 1 - \frac{\sum_i (y^{(i)} - \hat{y}^{(i)})^2}{\sum_i (y^{(i)} - \bar{y})^2}$$

- R^2 (coefficient of determination) close to 1 means the model explains most of the variance in the target; close to 0 means it explains almost none.

10 Overfitting, Underfitting, and Regularization

📖 Overfitting and Underfitting

Overfitting occurs when a model learns the training data too well, including its noise, and fails to generalize; underfitting occurs when a model is too simple to capture the underlying pattern at all.

10.1 The Bias-Variance Tradeoff

Aspect	High Bias (Underfitting)	High Variance (Overfitting)
Training Error	High	Low
Test Error	High	High
Model Complexity	Too simple	Too complex
Fix	Add features, more complex model	More data, regularization, simplify model

10.2 L1/L2 Regularization

💡 Penalizing Complexity

Regularization adds a penalty term to the cost function to discourage overly large weights, trading a little training accuracy for better generalization:

$$J_{L2}(w) = J(w) + \lambda \sum_j w_j^2 \quad (\text{Ridge})$$

$$J_{L1}(w) = J(w) + \lambda \sum_j |w_j| \quad (\text{Lasso})$$

L1 (Lasso) can shrink some weights to exactly zero, performing automatic feature selection; L2 (Ridge) shrinks weights smoothly without eliminating them.

10.3 Hyperparameter Tuning

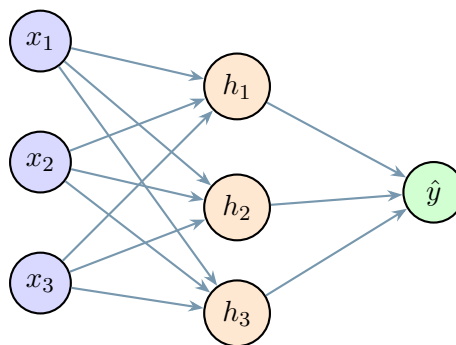
- **Hyperparameter:** A setting chosen before training (e.g., k in KNN, tree depth, learning rate α , regularization strength λ) — not learned from data directly.
- **Grid Search:** Exhaustively tries all combinations from a predefined set of hyperparameter values.
- **Random Search:** Samples random combinations, often finding good settings faster than grid search for large search spaces.

11 Introduction to Neural Networks

Artificial Neural Network

An artificial neural network is a model loosely inspired by biological neurons, composed of layers of simple computing units (neurons) connected by weighted links that are adjusted during training.

11.1 The Perceptron and Multi-Layer Perceptron



- **Perceptron:** A single neuron computes a weighted sum of inputs plus a bias, then applies an activation function: $\hat{y} = f(\sum_i w_i x_i + b)$.
- **Multi-Layer Perceptron (MLP):** Stacks an input layer, one or more **hidden layers**, and an output layer, allowing the network to model non-linear relationships.

11.2 Activation Functions

Function	Formula	Typical Use
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	Binary classification output layer
ReLU	$f(z) = \max(0, z)$	Hidden layers (fast, avoids vanishing gradient)
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	Multi-class classification output layer

11.3 Backpropagation

💡 How Networks Learn

Backpropagation computes how much each weight contributed to the final error by applying the chain rule of calculus backward through the network, then gradient descent updates every weight to reduce that error.

12 Introduction to Deep Learning

📖 Deep Learning

Deep learning uses neural networks with many hidden layers (“deep” architectures) to automatically learn hierarchical feature representations directly from raw data such as pixels or text.

12.1 Convolutional Neural Networks (CNNs)

💡 How CNNs Process Images

- **Convolution Layer:** Slides small filters (kernels) across the image to detect local patterns (edges, textures).
- **Pooling Layer:** Downsamples feature maps (e.g., max pooling) to reduce size and add translation invariance.
- **Fully Connected Layer:** Combines the extracted features to make the final classification.

CNNs dominate computer vision tasks: image classification, object detection, facial recognition.

12.2 Recurrent Neural Networks (RNNs)

📖 Recurrent Neural Network

An RNN processes sequential data (text, time series, audio) by maintaining a hidden state that is updated at each time step, allowing information from earlier steps to influence later predictions.

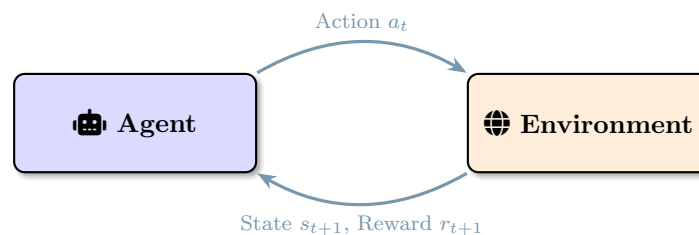
- **Vanishing Gradient Problem:** Standard RNNs struggle to learn long-range dependencies because gradients shrink as they propagate back through many time steps.
- **LSTM/GRU:** Gated variants of RNNs designed to retain relevant information over longer sequences.
- **Transfer Learning:** Reusing a large model pretrained on a general dataset and fine-tuning it on a smaller, task-specific dataset, saving time and data compared to training from scratch.

13 Reinforcement Learning Basics

Reinforcement Learning

Reinforcement Learning (RL) is a learning paradigm in which an **agent** learns to make decisions by taking **actions** in an **environment** to maximize cumulative **reward**.

13.1 Agents, Environments, and Rewards



- **Policy (π):** The agent's strategy mapping states to actions.
- **Reward Signal:** Immediate feedback (positive or negative) after an action.
- **Episode:** One complete run from a start state to a terminal state (e.g., one game).
- **Exploration vs. Exploitation:** The agent must balance trying new actions (exploration) against using known good actions (exploitation).

13.2 Q-Learning

Q-Learning

Q-Learning learns a table of action-values $Q(s, a)$ estimating the expected future reward of taking action a in state s , updated after every step using the Bellman equation.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- α (learning rate) controls how much new information overrides old estimates.
- γ (discount factor, $0 \leq \gamma \leq 1$) controls how much future rewards matter relative to immediate ones.

14 The Machine Learning Pipeline and Deployment

ML Pipeline

An ML pipeline is the automated sequence of steps — data ingestion, preprocessing, training, evaluation, and deployment — that turns raw data into a model serving real predictions in production.

14.1 From Notebook to Production

- **Experimentation:** Data scientists prototype models in notebooks (Jupyter) on a static dataset.

- **Productionization:** The winning model is wrapped in an API (e.g., a REST endpoint) so other applications can request predictions.
- **Batch vs. Real-Time Inference:** Batch scores many records periodically (e.g., nightly); real-time inference responds to individual requests within milliseconds.

14.2 Model Monitoring and Drift

💡 Why Deployed Models Need Monitoring

- **Data Drift:** The statistical properties of incoming data change over time relative to the training data.
- **Concept Drift:** The relationship between inputs and the correct output changes over time (e.g., customer behavior shifts).
- **Model Retraining:** Deployed models are periodically retrained on fresh data to stay accurate as drift accumulates.

15 Ethical and Responsible Machine Learning

📖 Why Ethics Matters in ML

Because ML models are trained on real-world data, they can inherit and amplify the biases present in that data, making responsible design and evaluation essential before deployment.

⚠️ Key Concerns

- **Bias and Fairness:** A model trained on biased historical data (e.g., biased hiring decisions) can learn and perpetuate that bias.
- **Explainability:** Complex models (deep neural networks, large ensembles) can be “black boxes,” making decisions hard to justify, especially in high-stakes domains (credit, health-care, criminal justice).
- **Privacy:** Training data may contain sensitive personal information that must be protected and anonymized.
- **Accountability:** It must be clear who is responsible when an ML system makes a harmful or incorrect decision.
- **Robustness:** Models can be fooled by adversarial examples — inputs deliberately perturbed to cause misclassification.

💡 Building Responsible ML Systems

- Audit training data and model outputs for disparate impact across demographic groups.
- Use interpretable models or explainability tools (e.g., feature importance, SHAP values) where decisions must be justified.
- Keep a human in the loop for high-stakes decisions rather than fully automating them.
- Document data sources, model assumptions, and limitations (“model cards”).

16 Core Mathematical Foundations for Machine Learning

Why Math Matters for ML

Linear algebra, probability, and calculus are the mathematical languages used to represent data, quantify uncertainty, and optimize model parameters in every ML algorithm.

16.1 Linear Algebra Essentials

- **Vector:** An ordered list of numbers representing a data point or a set of weights.
- **Matrix:** A 2D array of numbers, e.g., an entire dataset (rows = samples, columns = features).
- **Dot Product:** $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$, the core operation behind a neuron's weighted sum.
- **Eigenvectors/Eigenvalues:** Special directions/scales used by PCA to find directions of maximum variance.

16.2 Probability and Statistics Essentials

- **Conditional Probability:** $P(A | B) = \frac{P(A \cap B)}{P(B)}$, the basis of Bayes' Theorem and Naive Bayes.
- **Mean and Variance:** Measure the center and spread of a feature's distribution; used in standardization.
- **Bayes' Theorem:** $P(A | B) = \frac{P(B | A) P(A)}{P(B)}$.

16.3 Calculus and Optimization Essentials

- **Derivative:** Measures the rate of change of a function; the gradient generalizes this to multiple variables.
- **Gradient:** A vector of partial derivatives pointing in the direction of steepest increase of a function — gradient descent moves in the *opposite* direction to minimize cost.
- **Convex Function:** A function with a single global minimum (like MSE for linear regression), making optimization reliable.

17 Exam-Oriented Worked Examples

Worked Example 1: Linear Regression Prediction

A fitted model is $\hat{y} = 5 + 2x$. Predict $\hat{y}$ for $x = 7$:

$$\hat{y} = 5 + 2(7) = 19$$

Worked Example 2: One Gradient Descent Step

Given $w = 4$, gradient $\frac{\partial J}{\partial w} = 6$, and learning rate $\alpha = 0.1$:

$$w := 4 - 0.1(6) = 4 - 0.6 = 3.4$$

Worked Example 3: KNN Classification ($k = 3$)

A new point's 3 nearest neighbors have labels {Cat, Cat, Dog}. By majority vote, the new point is classified as **Cat** (2 out of 3 votes).

Worked Example 4: Entropy and Information Gain

A node has 10 examples: 6 positive, 4 negative:

$$\text{Entropy} = -\frac{6}{10} \log_2 \frac{6}{10} - \frac{4}{10} \log_2 \frac{4}{10} \approx 0.971$$

A pure split (all-positive / all-negative children) would give Information Gain ≈ 0.971 , the maximum possible for this node.

Worked Example 5: Confusion Matrix Metrics

Given $TP = 40$, $FP = 10$, $FN = 5$, $TN = 45$:

$$\text{Precision} = \frac{40}{40 + 10} = 0.80, \quad \text{Recall} = \frac{40}{40 + 5} \approx 0.889$$

$$F_1 = \frac{2(0.80)(0.889)}{0.80 + 0.889} \approx 0.842$$

Worked Example 6: One K-Means Update

Cluster points (2, 2), (4, 4), and (6, 4) are assigned to the same centroid. The new centroid is their mean:

$$\left(\frac{2 + 4 + 6}{3}, \frac{2 + 4 + 4}{3} \right) = (4, 3.33)$$

Worked Example 7: Naïve Bayes Classification

Classify a message with word “win” as spam/ham given $P(\text{spam}) = 0.3$, $P(\text{“win”} \mid \text{spam}) = 0.8$, $P(\text{ham}) = 0.7$, $P(\text{“win”} \mid \text{ham}) = 0.05$:

$$P(\text{spam}) \times P(\text{“win”} \mid \text{spam}) = 0.3 \times 0.8 = 0.24$$

$$P(\text{ham}) \times P(\text{“win”} \mid \text{ham}) = 0.7 \times 0.05 = 0.035$$

Since $0.24 > 0.035$, classify as **spam**.

Worked Example 8: Regression Error Metrics

Actual values $y = [3, 5, 2]$, predictions $\hat{y} = [2, 5, 4]$:

$$MAE = \frac{|3 - 2| + |5 - 5| + |2 - 4|}{3} = \frac{1 + 0 + 2}{3} = 1.0$$

$$MSE = \frac{1^2 + 0^2 + 2^2}{3} = \frac{5}{3} \approx 1.67, \quad RMSE \approx 1.29$$

18 Quick Revision Glossary

Term	One-Line Meaning
Machine Learning	Field enabling computers to learn patterns from data instead of fixed rules.
Supervised Learning	Learning a mapping from labeled input-output pairs.
Unsupervised Learning	Finding structure in unlabeled data.
Reinforcement Learning	Learning a policy via rewards from interacting with an environment.
Feature	A measurable input property used by a model.
Label	The known, correct output for a training example in supervised learning.
Feature Engineering	Creating new, more informative features from raw data.
Curse of Dimensionality	Data becomes sparse and harder to learn from as feature count grows.
Overfitting	Model fits training noise and fails to generalize to new data.
Underfitting	Model is too simple to capture the underlying pattern.
Bias-Variance Tradeoff	Balancing model simplicity (bias) against sensitivity to training data (variance).
Regularization	Penalty added to the cost function to discourage overly complex models.
Linear Regression	Predicts a continuous value as a weighted linear combination of features.
Logistic Regression	Predicts class probability via the sigmoid of a linear combination of features.
Gradient Descent	Iterative optimization that updates weights opposite to the cost gradient.
K-Nearest Neighbors (KNN)	Classifies a point by majority vote among its k closest training points.
Naive Bayes	Bayes'-Theorem-based classifier assuming conditional feature independence.
Decision Tree	Splits data on feature thresholds to reach a label at each leaf.
Entropy	Measure of impurity/disorder used to choose decision tree splits.
Support Vector Machine (SVM)	Finds the maximum-margin hyperplane separating classes.
Ensemble Learning	Combining multiple models' predictions for better accuracy/robustness.
Bagging	Trains models in parallel on bootstrapped samples and averages/votes results.

Term	One-Line Meaning
Random Forest	Bagged decision trees, each using a random subset of features per split.
Boosting	Trains models sequentially, each correcting the previous ones' errors.
K-Means Clustering	Iteratively assigns points to the nearest of k centroids and updates them.
Hierarchical Clustering	Builds a tree of clusters by repeatedly merging the closest pair.
DBSCAN	Density-based clustering that also detects noise/outlier points.
Principal Component Analysis (PCA)	Projects data onto directions of maximum variance to reduce dimensions.
Cross-Validation	Rotating train/validation splits (k -fold) for a more robust performance estimate.
Confusion Matrix	Table of TP/FP/FN/TN counts summarizing classification results.
Precision	Of predicted positives, the fraction that are actually positive.
Recall	Of actual positives, the fraction correctly predicted positive.
F1 Score	Harmonic mean of precision and recall.
ROC-AUC	Area under the ROC curve; summarizes performance across all thresholds.
RMSE	Root Mean Squared Error; regression error metric in the same units as the target.
Hyperparameter	A model setting chosen before training rather than learned from data.
Neural Network	Layers of connected neurons that learn weighted transformations of input data.
Activation Function	Non-linear function applied at each neuron (e.g., ReLU, Sigmoid, Softmax).
Backpropagation	Chain-rule-based algorithm for computing gradients through a neural network.
Convolutional Neural Network (CNN)	Deep network using convolution/pooling layers, dominant in computer vision.
Recurrent Neural Network (RNN)	Neural network with a hidden state for processing sequential data.
Q-Learning	Reinforcement learning method that learns action-values via the Bellman equation.
Data Drift	Change over time in the statistical properties of incoming data.

Term	One-Line Meaning
Model Card	Documentation of a model's data, assumptions, and limitations.
Data Leakage	Information from outside the training set unintentionally influences training.
One-Hot Encoding	Represents a categorical variable as binary indicator columns.
Grid Search	Exhaustive search over a predefined set of hyperparameter combinations.
Elbow Method	Plots within-cluster variance against k to help choose the number of clusters.
Exploration vs. Exploitation	The RL agent's tradeoff between trying new actions and using known good ones.
Transfer Learning	Fine-tuning a model pretrained on a large general dataset for a new, specific task.
Learning Rate	Step size α controlling how much weights change per gradient descent update.
Stratified k-Fold	Cross-validation variant that preserves class proportions in each fold.

19 Question Bank Blueprint (80 Marks Support)

💡 Suggested 80 Marks Distribution

- **Section A:** MCQs and definitions (20 marks)
- **Section B:** Short conceptual questions (20 marks)
- **Section C:** Descriptive/theory questions (20 marks)
- **Section D:** Numerical/problem-solving questions (20 marks)

19.1 Important Theory Questions

- Differentiate supervised, unsupervised, and reinforcement learning with examples.
- Explain the complete machine learning workflow from data collection to deployment.
- Differentiate feature scaling techniques: min-max scaling versus standardization.
- Explain gradient descent and the effect of the learning rate on convergence.
- Differentiate linear regression and logistic regression.
- Explain how K-Nearest Neighbors classifies a new point, and the effect of choosing k .
- Explain how a decision tree chooses the best feature to split on.
- Differentiate bagging and boosting as ensemble techniques.
- Explain the K-Means clustering algorithm and how the number of clusters is chosen.

- Explain Principal Component Analysis and why dimensionality reduction is useful.
- Explain why accuracy can be a misleading metric, and how precision/recall address this.
- Explain the bias-variance tradeoff and how it relates to overfitting and underfitting.
- Differentiate L1 and L2 regularization.
- Explain backpropagation and its role in training neural networks.
- Differentiate CNNs and RNNs in terms of the type of data they are best suited for.
- Explain the agent-environment loop in reinforcement learning and the Q-Learning update rule.
- Discuss ethical concerns in machine learning such as bias, explainability, and privacy.

19.2 Important Numerical/Analytical Questions

- Predict an output value given a fitted linear regression equation.
- Compute one gradient descent update given a weight, gradient, and learning rate.
- Classify a point using KNN given its nearest neighbors' labels.
- Compute entropy and information gain for a given split of examples.
- Compute precision, recall, and F1 score from a confusion matrix.
- Compute the updated centroid after one K-Means assignment step.
- Classify a message using Naive Bayes given simple probability tables.
- Compute MAE, MSE, and RMSE given actual and predicted values.

19.3 Sample MCQs with Answer Key

1. Which learning paradigm uses labeled input-output pairs?
(a) Unsupervised Learning (b) Supervised Learning (c) Reinforcement Learning (d) Semi-Supervised Learning
2. Which algorithm has no explicit training phase and classifies by distance to neighbors?
(a) Decision Tree (b) Naive Bayes (c) K-Nearest Neighbors (d) Logistic Regression
3. Which metric measures impurity used to choose decision tree splits?
(a) Entropy (b) Precision (c) RMSE (d) Learning Rate
4. Bagging primarily helps reduce which problem?
(a) Bias (b) Variance (c) Learning rate (d) Data drift
5. Which algorithm does not require specifying the number of clusters in advance?
(a) K-Means (b) Hierarchical Clustering (c) DBSCAN (d) PCA
6. Which metric is the harmonic mean of precision and recall?
(a) Accuracy (b) F1 Score (c) RMSE (d) R^2
7. Which regularization technique can shrink some weights to exactly zero?
(a) L1 (Lasso) (b) L2 (Ridge) (c) Dropout (d) Batch Normalization

8. Which neural network architecture is best suited to sequential data such as text or time series?
(a) CNN (b) RNN (c) Random Forest (d) SVM
9. In Q-Learning, what does the discount factor γ control?
(a) Learning speed (b) The weight given to future rewards (c) The number of actions
(d) The exploration rate only

💡 Answer Key

1–(b), 2–(c), 3–(a), 4–(b), 5–(c), 6–(b), 7–(a), 8–(b), 9–(b)

20 Summary

🔗 Key Takeaways

- Machine learning infers models directly from data, spanning supervised, unsupervised, and reinforcement learning paradigms.
- Careful data preprocessing and feature engineering underpin the performance of every downstream algorithm.
- Regression (linear/logistic), distance/probability-based methods (KNN, Naive Bayes), trees and margins (Decision Trees, SVM), and ensembles (bagging, boosting) form the core supervised toolkit.
- Clustering (K-Means, hierarchical, DBSCAN) and PCA reveal structure in unlabeled data and reduce dimensionality.
- Rigorous evaluation (cross-validation, confusion-matrix metrics, regression metrics) and awareness of the bias-variance tradeoff prevent overfitting/underfitting.
- Neural networks and deep learning (CNNs, RNNs) extend ML to images, text, and sequences via backpropagation.
- Reinforcement learning, production ML pipelines, and responsible/ethical practice complete the path from a trained model to a trustworthy, deployed system.